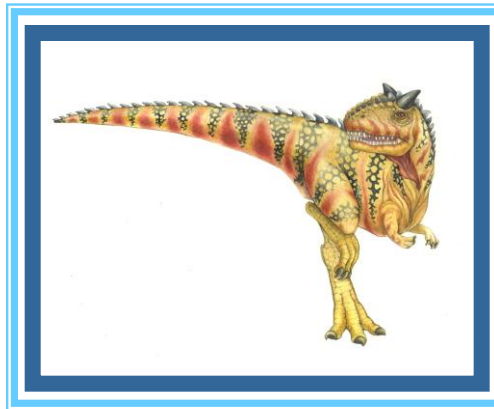
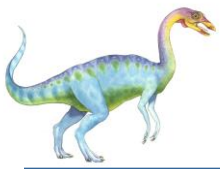


Chapter 6: Process Synchronization





Chapter 5: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores





Background

- ❑ Processes can execute concurrently
 - ❑ May be interrupted at any time, partially completing execution
- ❑ Concurrent access to shared data may result in data inconsistency
- ❑ Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- ❑ Illustration of the problem:

Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers. Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.





Bounded-Buffer – Producer

```
item next_produced;
while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```





Bounded Buffer – Consumer

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```



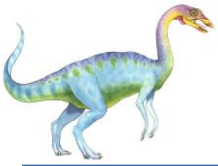
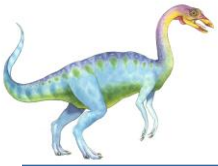


Illustration of the problem:

Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers.

Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.





Producer

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE) ;  
        /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```





Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```





Race Condition

- `counter++` could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- `counter--` could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with “counter = 5” initially:

S0: producer execute	<code>register1 = counter</code>	{register1 = 5}
S1: producer execute	<code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer execute	<code>register2 = counter</code>	{register2 = 5}
S3: consumer execute	<code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer execute	<code>counter = register1</code>	{counter = 6}
S5: consumer execute	<code>counter = register2</code>	{counter = 4}





Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- **Critical section problem** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**





Critical Section

- General structure of process P_i

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```





Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes





Critical-Section Handling in OS

- ❑ Two general approaches are used to handle critical sections in operating systems: (1) **preemptive kernels** and (2) **nonpreemptive kernels**.
- ❑ A **Preemptive** kernel allows a process to be preempted while it is running in kernel mode
- ❑ A **Non-preemptive** kernel does not allow a process running in kernel mode to be pre-empted
 - ❑ – runs until exits kernel mode, blocks, or voluntarily yields CPU
- ❑ Essentially free of race conditions in kernel mode as only one process is active in the kernel at a time
- ❑ A preemptive kernel is more suitable for real-time programming, as it will allow a real-time process to preempt a process currently running in the kernel.
- ❑ Furthermore, a preemptive kernel may be more responsive, since there is less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the processor to waiting processes.





Peterson's Solution

- ❑ a classic software-based solution to the critical-section problem
- ❑ no guarantees that Peterson's solution will work correctly on modern architectures.
- ❑ However, it provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting.
- ❑ restricted to two processes that alternate execution between their critical sections and remainder sections.
- ❑ The two processes share two variables:
 - ❑ `int turn;`
 - ❑ `Boolean flag[2]`
- ❑ The variable `turn` indicates whose turn it is to enter the critical section
- ❑ The `flag` array is used to indicate if a process is ready to enter the critical section. `flag[i] = true` implies that process P_i is ready!

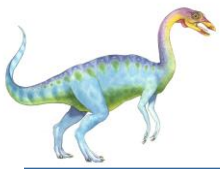




Algorithm for Process P_i

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while (true);
```





Peterson's Solution (Cont.)

Provable that the three CS requirement are met

1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.





Peterson's Solution (Cont.)

- Provable that the three CS requirement are met:

1. Mutual exclusion is preserved

P_i enters CS only if:

either `flag[j] = false` or `turn = i`

- if both processes can be executing in their critical sections at the same time, then **`flag[0] = flag[1] = true`**.
 - P_0 and P_1 could not have successfully executed their while statements at about the same time, since the value of `turn` can be either 0 or 1 but cannot be both
 - Hence, one of the processes —say, P_j —must have successfully executed the while statement, whereas P_i had to execute at least one additional statement ("`turn == j`").
 - However, at that time, `flag[j] = true` and `turn = j`, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved
- ## 2. Progress requirement is satisfied
- ## 3. Bounded-waiting requirement is met

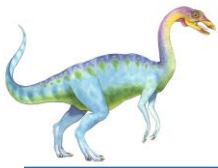




Peterson's Solution (Cont.)

- To prove **properties 2 and 3**, we note that a process P_i can be prevented from entering the critical section only if it is stuck in the while loop with the condition **flag[j] = true** and **turn = j**; this loop is the only one possible.
- If P_j is not ready to enter the critical section, then **flag[j] = false**, and P_i can enter its critical section.
- If P_j has set **flag[j]** to true and is also executing in its while statement, then either **turn = i** or **turn = j**
- If **turn == i**, then P_i will enter the critical section.
- If **turn == j**, then P_j will enter the critical section.
- However, once P_j exits its critical section, it will reset **flag[j] = false**, allowing P_i to enter its critical section.
- If P_j resets **flag[j]** to true, it must also set **turn** to i .
- Thus, since P_i does not change the value of the variable **turn** while executing the while statement, P_i will enter the critical section (**progress**) after at most one entry by P_j (**bounded waiting**).

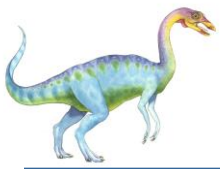




Synchronization Hardware

- ❑ Many systems provide hardware support for implementing the critical section code.
- ❑ All solutions below based on idea of **locking**
 - ❑ Protecting critical regions via locks
 - ❑ a process must acquire a lock before entering a critical section;
 - ❑ it releases the lock when it exits the critical section.





Solution to Critical-section Problem Using Locks

```
do {  
    acquire lock  
        critical section  
    release lock  
    remainder section  
} while (TRUE);
```





- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - ▶ message is passed to all the processors.
 - ▶ This message passing delays entry into each critical section, and system efficiency decreases.
 - ▶ Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - ▶ **Atomic** = non-interruptible
 - Either test memory word and set value
 - Or swap contents of two memory words





test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target; //Store the old value
    *target = TRUE; //Set the target value
    return rv; //Return the old value
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.





Solution using test_and_set()

- Shared Boolean variable lock, initialized to FALSE
- Solution:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
    lock = false;  
        /* remainder section */  
} while (true);
```





Solution using test_and_set() Cont..

❑ Mutual exclusion is ensured

- ❑ it always returns whatever value is sent to it and sets lock to true.
- ❑ The first process will enter the critical section at once as TestAndSet(lock) will return false and it'll break out of the while loop.
- ❑ The other processes cannot enter now as lock is set to true and so the while loop continues to be true.

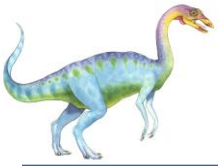
❑ Progress is also ensured.

- ❑ Once the first process gets out of the critical section, lock is changed to false.
- ❑ So, now the other processes can enter one by one.

❑ Bounded waiting is not ensured.

- ❑ After the first process, any process can go in.
- ❑ There is no queue maintained, so any new process that finds the lock to be false again can enter.





Swap

```
void Swap(boolean *a, boolean *b) {  
    boolean temp = *a;  
    *a = *b;  
    *b = temp;  
}
```





Mutual-exclusion implementation with the Swap() instruction.

- A global Boolean variable lock is declared and is initialized to false.
- In addition, each process has a local Boolean variable key
- The structure of process P_i is as shown

```
do {  
    key = TRUE;  
    while (key == TRUE)  
        Swap(&lock, &key);  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```





Mutual-exclusion implementation with the Swap() instruction. Contd...

- ❑ key is set to true and then swapped with lock.
- ❑ First process will be executed, and in while(key), since key=true , swap will take place and hence lock=true and key=false.
- ❑ Again next iteration takes place while(key) but key=false , so while loop breaks and first process will enter in critical section.
- ❑ Now another process will try to enter in Critical section, so again key=true and hence while(key) loop will run and swap takes place so, lock=true and key=true (since lock=true in first process).
- ❑ Again on next iteration while(key) is true so this will keep on executing and another process will not be able to enter in critical section.
- ❑ **Mutual exclusion is ensured.**
- ❑ Again, out of the critical section, lock is changed to false, so any process finding it gets to enter the critical section. **Progress is ensured.**
- ❑ However, again bounded waiting is not ensured for the very same reason.





Bounded-waiting Mutual Exclusion with test_and_set

algorithm using the TestAndSet() instruction that satisfies all the critical-section requirements

boolean waiting[n];

boolean lock;

These data structures are initialized to false.

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;
    /* remainder section */
} while (true);
```





Bounded-waiting Mutual Exclusion with test_and_set Contd...

- To prove that the mutual exclusion requirement is met,
- Process P_i can enter its critical section only if either $\text{waiting}[i] == \text{false}$ or $\text{key} == \text{false}$.
- The value of key can become false only if the $\text{TestAndSet}()$ is executed.
- The first process to execute the $\text{TestAndSet}()$ will find $\text{key} == \text{false}$; all others must wait.
- The variable $\text{waiting}[i]$ can become false only if another process leaves its critical section;
- only one $\text{waiting}[i]$ is set to false, maintaining the mutual-exclusion requirement





Semaphore

- Synchronization tool that provides more sophisticated ways) for process to synchronize their activities.
- Semaphore **S** – integer variable
- Can only be accessed via two indivisible (atomic) operations
 - **wait()** and **signal()** Originally called
 - ▶ **P()** – (from the Dutch *proberen*, “to test”) and **V()** – (from *verhogen*, “to increment”)

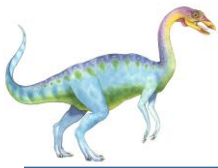
- Definition of the **wait()** operation

```
wait(S) {  
    while (S <= 0)  
        ; // no-op  
    S--;  
}
```

- Definition of the **signal()** operation

```
signal(S) {  
    S++;  
}
```





Requirements

- All modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly.
- That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.
- In addition, in the case of wait(S), the testing of the integer value of S ($S \leq 0$), as well as its possible modification ($S--$), must be executed without interruption





Semaphore Usage

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1

- Same as a **mutex lock**

- mutex, initialized to 1

do {

 wait(mutex);

 // critical section

 signal(mutex);

 // remainder section }

while (TRUE);

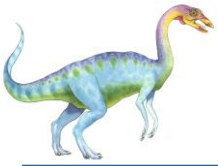




Counting semaphores

- ❑ can be used to control access to a given resource consisting of a finite number of instances.
- ❑ The semaphore is initialized to the number of resources available.
- ❑ Each process that wishes to use a resource performs a `wait()` operation on the semaphore (thereby decrementing the count).
- ❑ When a process releases a resource, it performs a `signal()` operation (incrementing the count).
- ❑ When the count for the semaphore goes to 0, all resources are being used.
- ❑ After that, processes that wish to use a resource will block until the count becomes greater than 0.





- Can solve various synchronization problems
- Consider P_1 and P_2 that require S_1 to happen before S_2

Create a semaphore “**synch**” initialized to 0

P1:

$S_1;$

signal(synch) ;

P2:

wait(synch) ;

$S_2;$

- Can implement a counting semaphore **S** as a binary semaphore





Semaphore Implementation

- ❑ Must guarantee that no two processes can execute the **wait()** and **signal()** on the same semaphore at the same time
- ❑ Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
 - ❑ Could now have **busy waiting** in critical section implementation
 - ▶ But implementation code is short
 - ▶ Little busy waiting if critical section rarely occupied
- ❑ Note that applications may spend lots of time in critical sections and therefore this is not a good solution
- ❑ To overcome the need for busy waiting, modify the definition of the wait() and signal() semaphore operations.
- ❑ When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait.
- ❑ Rather than engaging in busy waiting, the process can block itself.
- ❑ The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- ❑ Then control is transferred to the CPU scheduler, which selects another process to execute





Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
- `typedef struct{
 int value;
 struct process *list;
} semaphore;`





Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

semaphore values may be negative, although semaphore values are never negative under the classical definition of semaphores with busy waiting.

If a semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.





Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0
`wait(S);`
`wait(Q);`
...
`signal(S);`
`signal(Q);`

P_1
`wait(Q);`
`wait(S);`
...
`signal(Q);`
`signal(S);`

P_0 and P_1 are deadlocked

a set of processes is in a deadlock state when every process in the set is waiting for an event that can be caused only by another process in the set.

- **Starvation – indefinite blocking**
 - A process may never be removed from the semaphore queue in which it is suspended – if list is accessed in LIFO order





Priority Inversion

- ❑ Scheduling problem when lower-priority process holds a lock needed by higher-priority process
- ❑ The situation becomes more complicated if the lower-priority process is preempted in favor of another process with a higher priority.
- ❑ **Example:** assume there are three processes, L, M, and H, whose priorities follow the order $L < M < H$.
- ❑ Assume that process H requires resource R, which is currently being accessed by process L.
- ❑ Process H would wait for L to finish using resource R.
- ❑ Now suppose that process M becomes runnable, thereby preempting process L
- ❑ Indirectly, a process with a lower priority—process M—has affected how long process H must wait for L to relinquish resource R.
- ❑ This problem is known as **priority inversion**.

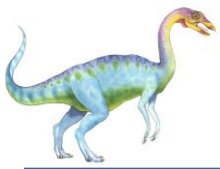




Priority-inheritance protocol

- **Priority inversion problem** Solved via **priority-inheritance protocol**
- all processes that are accessing resources needed by a higher-priority process inherit the higher priority until they are finished with the resources in question.
- When they are finished, their priorities revert to their original values.
- In the previous example, a priority-inheritance protocol would allow process L to temporarily inherit the priority of process H, thereby preventing process M from preempting its execution.
- When process L had finished using resource R, it would relinquish its inherited priority from H and assume its original priority.
- Because resource R would now be available, process H—not M—would run next





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem





Bounded-Buffer Problem

- n buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n





Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
} while (true);
```





Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
do {  
    wait(full);  
    wait(mutex);  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true);
```





Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - Readers – only read the data set; they do **not** perform any updates
 - Writers – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities
- first readers–writers problem- requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object. In other words, no reader should wait for other readers to finish simply because a writer is waiting.
- The second readers –writers problem requires that, once a writer is ready, that writer perform its write as soon as possible. In other words, if a writer is waiting to access the object, no new readers may start reading
- A solution to either problem may result in starvation. In the first case, writers may starve; in the second case, readers may starve





Readers-Writers Problem (Cont.)

□ Shared Data

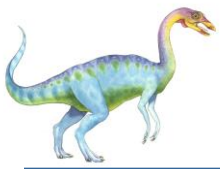
- solution to the first readers–writers problem, the reader processes share the following data structures

semaphore **mutex**, **wrt**;

int **readcount**;

- The semaphore - **wrt** is common to both reader and writer processes.
 - ▶ **wrt** functions as a mutual-exclusion semaphore for the writers
 - ▶ also used by the first or last reader that enters or exits the critical section.
 - ▶ It is not used by readers who enter or exit while other readers are in their critical sections.
- The **mutex** semaphore is used to ensure mutual exclusion when the variable **readcount** is updated.
 - The **readcount** variable keeps track of how many processes are currently reading the object.





The structure of a writer process

```
do {  
    wait(wrt);  
    ...  
    /* writing is performed */  
    ...  
    signal(wrt);  
} while (true);
```





Readers-Writers Problem (Cont.)

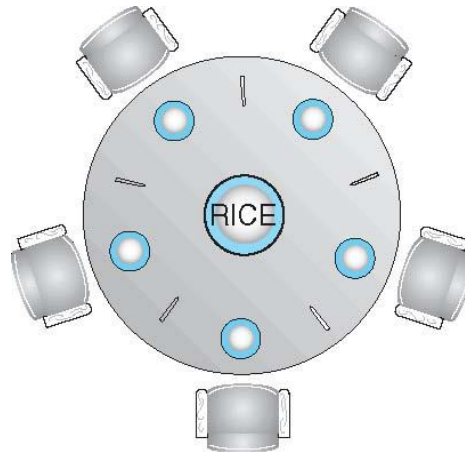
- The structure of a reader process

```
do {  
    wait(mutex);  
    readcount++;  
    if (readcount == 1)  
        wait(wrt);  
    signal(mutex);  
  
    ...  
    /* reading is performed */  
  
    ...  
    wait(mutex);  
    read count--;  
    if (readcount == 0)  
        signal(wrt);  
    signal(mutex);  
} while (true);
```





Dining-Philosophers Problem



- ❑ Philosophers spend their lives alternating thinking and eating
- ❑ Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - ❑ Need both to eat, then release both when done
- ❑ classic synchronization problem





Dining-Philosophers Problem Algorithm

- In the case of 5 philosophers
 - ▶ Semaphore **chopstick** [5] initialized to 1

- The structure of Philosopher *i*:

```
do {  
    wait (chopstick[i] );  
    wait (chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

- What is the problem with this algorithm?





- this solution guarantees that no two neighbors are eating simultaneously
- Nevertheless must be rejected because it could create a deadlock.
- Suppose that all five philosophers become hungry simultaneously and each grabs her left chopstick.
- All the elements of chopstick will now be equal to 0.
- When each philosopher tries to grab right chopstick - will be delayed forever





Dining-Philosophers Problem Algorithm (Cont.)

- Deadlock handling
 - Allow at most 4 philosophers to be sitting simultaneously at the table.
 - Allow a philosopher to pick up the chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section).
 - Use an asymmetric solution -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.





Problems with Semaphores

- ❑ Incorrect use of semaphore operations:
 - ❑ signal (mutex) wait (mutex)
 - ❑ wait (mutex) ... wait (mutex)
 - ❑ Omitting of wait (mutex) or signal (mutex) (or both)
- ❑ Deadlock and starvation are possible.



End of Chapter 6

